Reporte

Nombre: Equipo 6 Luis David Cigala Montes Jared Alejandro Garcia Alcocer Gabriel Sánchez Rivera	**Matrícula**: 2120993 2119041 2121131
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Actividad 4, Preguntas del reporte (TechStore)
Fecha: 10/28/26	
Bibliografia (Formato APA): Archivo de la actividad 4 brindada por el ingeniero	

Objetivo:

Responder las cinco preguntas del reporte de la actividad 4 explicando, con el código de TechStore, las decisiones de diseño: Decimal, unidades, Specification, Domain Events y reutilización.

Procedimiento:

Respuestas a las preguntas del reporte

1. ¿Por qué se usa Decimal en lugar de float para precios?

Float guarda los números en binario y no puede representar exactamente valores como 0.1. Por ejemplo, 0.1 + 0.2 da 0.30000000000000004. En dinero, esos errores mínimos pueden cambiar el resultado de comparaciones como "mayor a 500" o "mayor a 5000", que son reglas de descuento de TechStore.

En el código, la clase Money convierte el monto con Decimal(str(amount)) y rechaza valores negativos o no finitos. Con Decimal, los cálculos de subtotal, total y descuento son exactos, y un total de exactamente 500.00 se compara correctamente contra la regla "mayor que 500".

2. ¿Cuál es la diferencia entre contar renglones y sumar unidades?

Un renglón es una línea del pedido (un producto con su cantidad). Las unidades son la suma de las cantidades de todos los renglones. En Order, la propiedad units suma item.quantity de cada renglón, y no usa la cantidad de renglones.

Esto importa porque el límite de 20 artículos y la regla del 10% (al menos 4 unidades) se calculan con unidades. Un solo renglón con 25 piezas rebasa el límite aunque sea un único renglón, y el pedido O001 tiene 2 renglones pero 4 unidades, por lo que cumple la regla de las 4 unidades.

3. ¿Qué ventaja tiene encapsular una regla en Specification?

La regla queda con nombre y en un solo lugar, por ejemplo TenPercentDiscountSpecification y FivePercentDiscountSpecification. Se puede evaluar sin modificar el pedido, se prueba por separado y se combina con otras condiciones.

Si cambia un umbral, como los 500 pesos, solo se modifica esa clase y no cada servicio donde se usaba. La función get_discount_rate solo pregunta qué regla se cumple, sin repetir las condiciones del descuento.

4. ¿Por qué los eventos se registran solo después de una transición válida?

Porque un evento afirma que algo ya sucedió. Si se registrara antes de validar, otros sistemas (inventario, correo o facturación) podrían reaccionar a un hecho que nunca ocurrió, por ejemplo avisar que un pedido fue enviado cuando en realidad se rechazó.

En el código, cada método (confirm, ship, cancel, deliver y return_order) valida primero, cambia el estado y agrega el evento al final. Si una validación falla, se lanza la excepción y el evento no se crea. Se comprobó con ship("") sobre un pedido en DRAFT: el estado sigue en DRAFT y la lista de eventos queda vacía.

5. ¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?

Casi todo. Se reutilizan domain.py (Money, Customer, OrderItem, Order, estados, eventos y excepciones), specifications.py, cargar_csv.py y las pruebas de dominio y de escenarios (test_domain.py y test_escenarios.py), que no importan FastAPI.

Solo habría que reescribir api.py (las rutas y la conversión de excepciones a códigos HTTP) y test_api.py. Esto es posible porque las reglas viven en el dominio y no en los controladores.